

Assignment 2-Case Study

Stage 4 Lab Activities

Implementing the SmartCare Domain Layer

DESIGN FIRST -> AI PAIR PROGRAMMING -> REVIEW -> VERIFY | 1hour

A - Revisit Approved UML

The approved model consists of Patient, Practitioner, and Appointment classes, with one to many relationships connecting the patient and practitioner to their appointments as stated in the previous documentation.

B - Implement Patient: AI OFF

class Patient:

```
def __init__(self, patient_id: int, name: str, contact_info: str):
```

```
    # Basic validation: check if the name is blank
```

```
    if name == "":
```

```
        raise ValueError("Name cannot be empty!")
```

```
    self.id: int = patient_id
```

```
    self.name: str = name
```

```
    self.contact_info: str = contact_info
```

```
    self.appointments: list = []
```

```
def get_history(self) -> list:
```

```
    return self.appointments
```

C - Implement Practitioner: AI OFF

```
class Practitioner:

    def __init__(self, practitioner_id: int, name: str, specialty: str):

        self.id: int = practitioner_id

        self.name: str = name

        self.specialty: str = specialty

        self.appointments: list = []

    def get_schedule(self) -> list:

        return self.appointments
```

D - Implement Appointment: AI ON

```
from enum import Enum

# Create a simple Enum for the status

class AppointmentStatus(Enum):

    SCHEDULED = "Scheduled"

    CANCELLED = "Cancelled"

class Appointment:

    def __init__(self, appointment_id: int, date: str, time: str, patient: Patient, practitioner: Practitioner):

        self.id: int = appointment_id

        self.date: str = date

        self.time: str = time

        self.patient: Patient = patient

        self.practitioner: Practitioner = practitioner

        self.status: AppointmentStatus = AppointmentStatus.SCHEDULED
```

```
def cancel(self) -> None:

    # Protect status transition: make sure it isn't already cancelled

    if self.status == AppointmentStatus.CANCELLED:

        raise Exception("This appointment is already cancelled!")

    self.status = AppointmentStatus.CANCELLED
```

E - Review Generated Code

Check model consistency, unsupported features, public state mutation, unnecessary inheritance, invented dependencies and error handling.

F - Manual Behaviour Checks

1. Create valid objects

```
p1 = Patient(1, "Jane Doe", "555-0100")

dr1 = Practitioner(101, "Dr. Smith", "General Practice")

appt = Appointment(1, "Oct 15", "10:00 AM", p1, dr1)
```

2. Cancel a scheduled appointment

```
appt.cancel()

print("Success: The appointment was cancelled.")
```

3. Attempt an illegal repeated transition

```
try:

    appt.cancel() # Trying to cancel it again

except Exception as error:

    print("Error caught:", error)
```

4. Test invalid input (empty name)

try:

```
p2 = Patient(2, "", "123-4567")
```

except ValueError as error:

```
print("Error caught:", error)
```

G – Refactor

The code was reviewed to remove unnecessary code and make the implementation simpler and be structurally consistent. The AI's output was already highly constrained, so no further removal of UI or database elements was needed.

H - AI Engineering Log

Prompt: "Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML."

Generated contribution: The Appointment class, AppointmentStatus Enum, and IllegalTransitionError exception logic.

Decisions: Accepted the use of a private `_status` attribute with a specific `cancel()` method to protect state transitions.

Verification: Verified via the manual behaviour checks script that an already-cancelled appointment raises an exception rather than silently failing.

Suggested AI prompt

Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML.

Reflection

During generation, I rejected any AI tendency to add .save() methods or database connection strings, as the instructions specifically restricted the scope to the domain layer. The approved design heavily limited the AI by providing exact class names, forcing the use of type hints, and explicitly forbidding service classes (like notifications or UI interfaces). This ensured the final code remained a pure representation of the business logic.